BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 04

> **Objective & Lecture Mapping:** In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment[cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

> Informed search algorithms require a heuristic function, h(n), which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called Week_04 in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class[cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula h(n) = |x_1 - x_2| + |y_1 - y_2| and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

> A* evaluates nodes by combining the path cost so far, g(n), and the estimated cost to the goal, h(n). The evaluation function is f(n) = g(n) + h(n).

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses g(n), your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, g(n) = 0. Calculate h(n) using your chosen heuristic method, and set f(n) = g(n) + h(n).
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate g_{new} = g_{current} + 1, h_{new} using your heuristic, and f_{new} = g_{new} + h_{new}. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

> Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

IT24103933

Uniform-Cost Search (UCS) and AI Search comes from how each algorithm evaluates nodes. UCS selects the next node to expand based only on the total path cost accumulated from the start, g(n). Because it does not use any information about how close a node is to the goal, UCS tends to expand outward evenly in all directions until it eventually reaches the goal.

AI Search adds an additional component: a heuristic estimate of the remaining cost to reach the goal, h(n). It combines both values into f(n) = g(n) + h(n). This allows AI to focus its search toward the goal instead of exploring every direction uniformly. Nodes that appear to lead away from the goal receive higher f(n) values and are expanded later or ignored.

As a result, AI usually finds the goal much faster because the heuristic guides the search, while UCS may explore many unnecessary paths before reaching the optimal solution.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

Manhattan Distance is admissible on a 4-direction grid because it never gives a value larger than the actual shortest path. In a grid where movement is limited

to Up, Down, Left, and Right, the minimum number of steps required to reach the goal ignoring obstacle is exactly the Manhattan Distance. Any walls or blocked cells can only make the real path longer, not shorter. Therefore, Manhattan Distance always stays below or equal to the true cost, which satisfies the admissibility requirement for heuristics.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

If diagonal movement is permitted and each diagonal step costs 1, then Manhattan Distance is no longer an admissible heuristic. This is because Manhattan Distance counts horizontal and vertical movement separately, treating a diagonal move as two steps. In reality, a diagonal step reaches both directions at once for a cost of 1, so Manhattan Distance would give a value larger than the true shortest path. Any heuristic that overestimates the real cost cannot be admissible.

In an 8-direction grid, the correct admissible heuristic is the Chebyshev Distance, defined as

$$\max(|x_1 - x_2|,\ |y_1 - y_2|)$$

This measure accurately reflects the minimum number of steps needed when diagonal moves are allowed at equal cost. If diagonal moves have a different cost (for example, more expensive than straight moves), then Euclidean Distance becomes the appropriate choice.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

A more effective heuristic for collecting all remaining food items is to account for the overall structure of the food layout rather than focusing only on the nearest piece. One strong option is to compute the **Minimum Spanning Tree (MST)** over all uneaten food positions.

Under this approach, the heuristic value $h(n)$ can be defined as:

- the distance from the agent to the closest food item, plus
- the total length of the MST that connects all remaining food items.

This works because any complete solution must at least cover the MST distance to visit every food location, and it must also pay the cost of reaching the first piece of food. Since the MST represents the minimum possible distance needed to connect all food points, this heuristic forms a valid lower bound and remains admissible. It also provides a much tighter estimate than simply measuring the distance to the nearest food, helping the search avoid short-sighted choices and plan more efficiently across the entire grid.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation